

Classifying newswires: a multiclass classification example

Multiclass Classification Overview

เมื่อมีมากกว่า 2 คลาส

- ก่อนหน้า: **binary classification** (positive vs negative)
- ตอนนี้: ต้องจำแนก 46 คลาสที่ไม่ทับซ้อนกัน
- ปัญหานี้คือ
single-label, multiclass classification

Reuters Dataset

ข้อมูลข่าวสั้น 46 หัวข้อ (Reuters, 1986)

- ข่าวสั้น + หมวดหมู่ของข่าว
- 46 topics
- ตัวอย่างอย่างน้อย 10 ข่าวต่อ topic
- Keras เตรียม dataset นี้ไว้ให้ใช้งานแล้ว

```
from keras.datasets import reuters  
  
(train_data, train_labels), (test_data, test_labels) =  
reuters.load_data(num_words=10000)
```

Reuters Dataset

```
len(train_data)
```

```
len(test_data)
```

```
train_data[10]
```

Data format

```
[
    [10, 25, 67],      # "I love cats"
    [122, 5, 990, 72]  # "Deep learning is fun"
]
```

เนื้อหาของข่าวถูกแทนด้วย **list** ของตัวเลข (word indices)

`train_data[10]`

`[1, 245, 273, 207, 156, ...]`

- ตัวเลขแต่ละตัว = index of word
- `num_words = 10000` → เก็บเฉพาะ 10,000 คำที่พบบ่อยที่สุด
- เช่นเดียวกับ IMDB dataset

Decoding Back to Text เหมือน IMDB

แปลงตัวเลขกลับเป็นคำได้เพื่อดูความหมาย

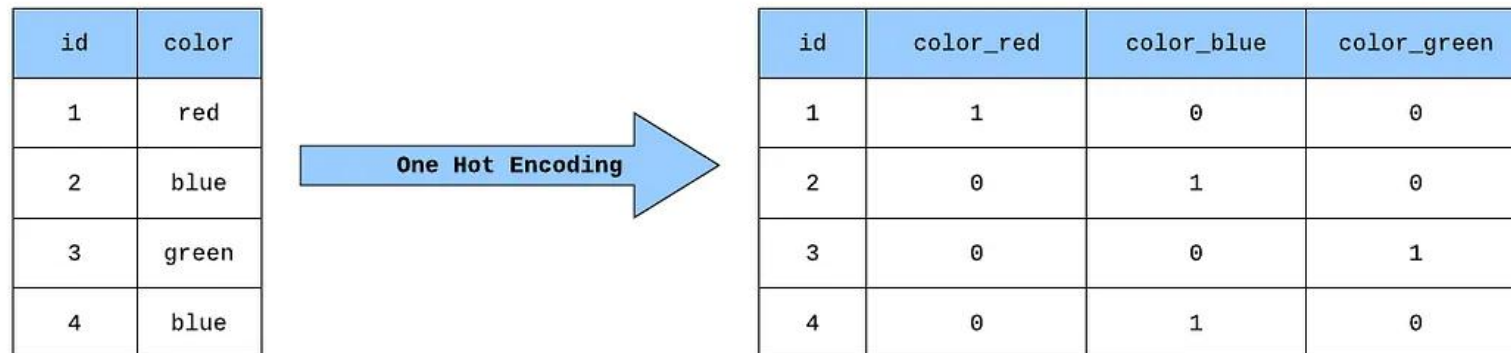
```
word_index = reuters.get_word_index()
reverse_word_index = {value: key for key, value in word_index.items()}
decoded_newswire = ' '.join(reverse_word_index.get(i - 3, '?') for i in
train_data[0])
```

Labels in Reuters Dataset

Labels เป็น integer ระหว่าง 0–45

```
train_labels[10]  
3
```

- หมายเลขนี้คือ **topic index**
- ใช้ในการสร้าง **one-hot vectors** เพื่อที่จะ train model
- แต่ละข่าวมี **topic** เพียงหนึ่งเดียว (single-label)



ขั้นตอนการทำงานคร่าวๆ

ขั้นตอนการทำงานคร่าวๆ

เดาสี อย่าเพิ่งดูหน้าถัดไป เดาก่อน

ขั้นตอนการทำงานคร่าวๆ

ขั้นตอนการทำงานคร่าวๆ

- Load data ข่าว (ตัวเลขล้วน)
- แปลง labels เป็น one-hot (46-dim)
- สร้าง network ด้วย softmax output บน output layer
- loss function ใช้ categorical_crossentropy
- Train + validate
- Predict และวัด accuracy

ตัวอย่างปัญหา: single-label vs multilabel

ประเภท

ความหมาย

single-label multiclass

ข่าว 1 ข่าวอยู่ได้แค่ 1 หัวข้อ (gd,nj Reuters)

multilabel multiclass

ข่าว 1 ข่าวอาจอยู่หลายหัวข้อ เช่น “economy + trade”

Preparing the Data

Preparing the Data

- แปลงข่าว (list ของตัวเลขคำ) → เวกเตอร์ที่ขนาดเท่ากันโมเดลนำไปใช้ได้
- Vectorize ทั้ง input sequences และ labels
- ใช้ one-hot encoding สำหรับ multiclass (46 classes)

```
[  
    [10, 25, 67],          # "I love cats"  
    [122, 5, 990, 72]      # "Deep learning is fun"  
]
```

```
index:    0 1 2 3 4 5 6 7 8 ...  
vector: [0 0 0 1 0 1 0 1 0 ...]
```

Code `x_train` and `x_test`

```
def vectorize_sequences(sequences, dimension=10000):  
    results = np.zeros((len(sequences), dimension))  
    for i, sequence in enumerate(sequences):  
        results[i, sequence] = 1.  
    return results
```

```
x_train = vectorize_sequences(train_data)  
x_test = vectorize_sequences(test_data)
```

Preparing the Labels

```
label: 3  
vector: [0 0 1 0 0 0 0 0 0 ... 0] # len = 46
```

Label มี 46 คลาส (topics) มีสองวิธีในการแปลง label:

- Integer tensor เช่น 3, 12, 45
- One-hot encoding (เวกเตอร์ขนาด 46)

one-hot labels

```
def to_one_hot(labels, dimension=46):  
    results = np.zeros((len(labels), dimension))  
    for i, label in enumerate(labels):  
        results[i, label] = 1.  
    return results
```

```
one_hot_train_labels = to_one_hot(train_labels)  
one_hot_test_labels = to_one_hot(test_labels)
```

Using Keras Built-in One-Hot Encoding

Keras มีฟังก์ชัน `to_categorical`

- แทน `to_one_hot()` ที่เขียนเอง
- ใช้ง่ายกว่า และลดความผิดพลาด

```
from keras.utils.np_utils import to_categorical
```

```
one_hot_train_labels = to_categorical(train_labels)  
one_hot_test_labels = to_categorical(test_labels)
```

Key Idea: Why the Network Must Change

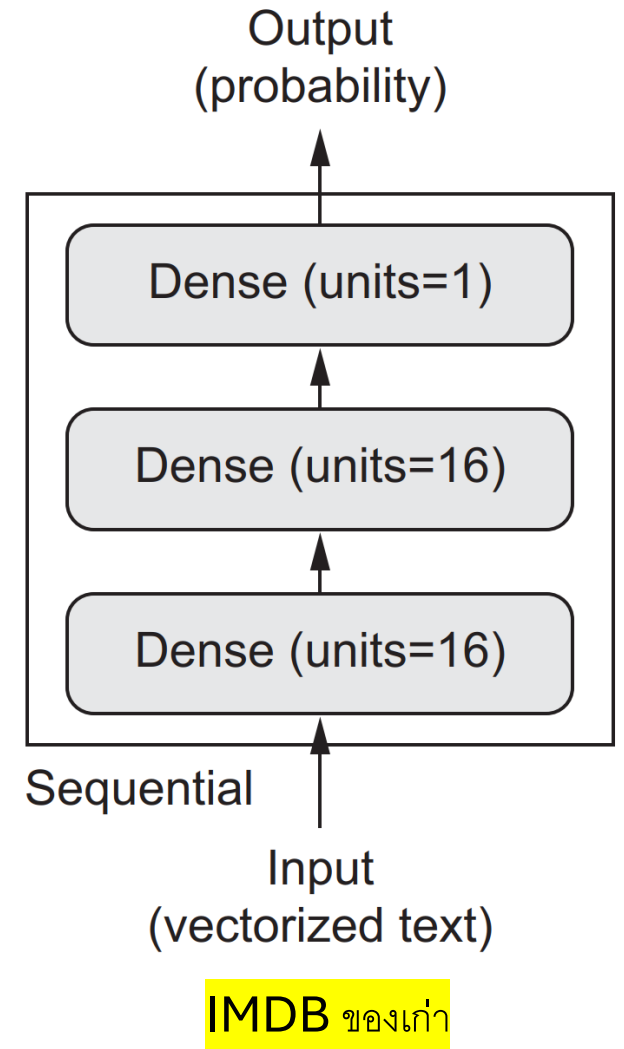
Topic classification (46 classes) $\neq$ Binary classification (2 classes)

- จำนวนคลาสเพิ่มจาก 2 classes $\rightarrow$ 46 classes
- Output space ใหญ่ขึ้นมาก
- ต้องการ hidden layers ที่มีมิติมากขึ้น
- เลเยอร์เล็ก ๆ อาจเกิด information bottleneck

Information Bottleneck Problem

Dense layers ต่อกันเป็นสาย

- แต่ละเลเยอร์รับข้อมูลจากเลเยอร์ก่อนหน้า
- ถ้าเลเยอร์กลางทิ้งข้อมูล → เลเยอร์หลังไม่สามารถกู้คืนได้
- เลเยอร์เล็ก (เช่น **16 units**) อาจจำกัดเกินไป
- ต้องใช้เลเยอร์ใหญ่ขึ้นเพื่อรองรับ **46 classes**



Model Architecture

Network

```
model = models.Sequential()  
model.add(layers.Dense(64, activation='relu', input_shape=(10000,)))  
model.add(layers.Dense(64, activation='relu'))  
model.add(layers.Dense(46, activation='softmax'))
```

Structure

- 2 hidden layers × 64 units
- Output = 46 (หนึ่งต่อหนึ่งกับหัวข้อข่าว)
- Activation = softmax

Softmax

Output vector $z = [z_1, z_2, z_3, \dots, z_n]$

$$\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_{i=1}^N \exp(z_i)}$$

- $\sum_{i=1}^N \text{Softmax}(z_i) = 1$

Ex

$Z = [2, 1, 0]$ คำนวณ soft max:

- $e^2 =$

- $e^1 =$

- $e^0 =$

Dense(46) + Softmax

Dense(46)

- แต่ละหัวข้อข่าวมี index ตั้งแต่ 0-45
- Output vector ยาว 46
- $\text{output}[i]$ = ความน่าจะเป็นที่ข่าวเป็น topic i

Softmax activation

- แปลงค่าดิบเป็น probability
- ผลรวม = 1
- เหมาะสำหรับ multiclass classification

Choosing the Loss Function

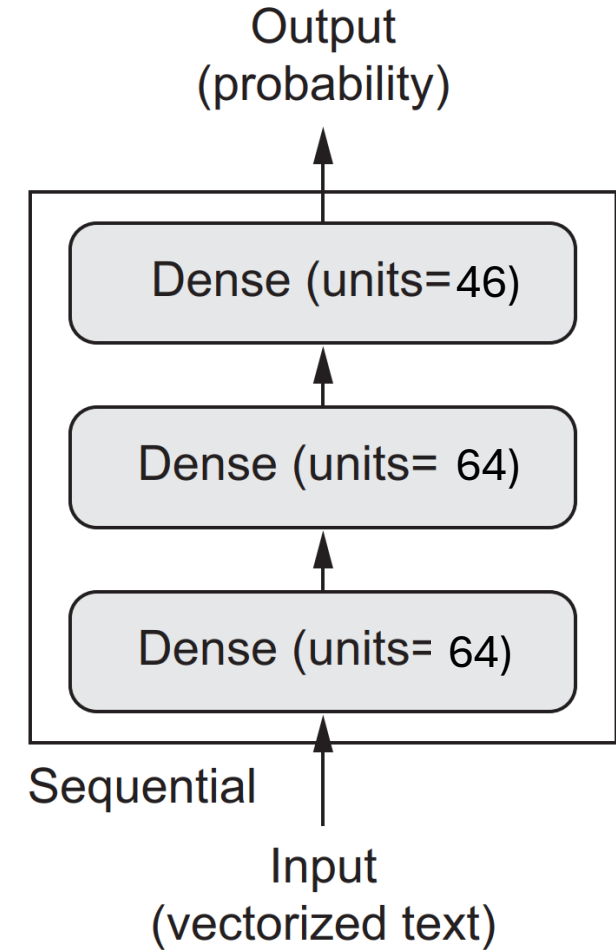
categorical_crossentropy

- ใช้เทียบ **distribution** ที่โมเดลทำนาย กับ **distribution** ของ **labels (one-hot)**
- ทำให้โมเดลเรียนรู้ให้ผลลัพธ์ใกล้เคียง **label** ที่แท้จริงที่สุด
- เป็น **loss** ที่เหมาะสมที่สุดสำหรับ **softmax output**

Compiling the Model

```
model.compile(optimizer='rmsprop',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

- **optimizer: rmsprop** ทำงานดีสำหรับปัญหาแบบ **text**
- **loss: categorical_crossentropy** → เหมาะกับ **multiclass**
- **metric: accuracy** → วัดว่าทำนายถูกกี่ครั้ง



network

Reuters Dataset เป้าหมายคือ ให้โมเดลทำนายหัวข้อข่าวจากข้อความ

- ปัญหานี้คือ single-label, multiclass classification
- ใช้ hidden layers ที่มีมิติใหญ่พอ (64 units)
- Output layer = 46 units + softmax
- Loss = categorical_crossentropy

Validation Set

เหตุผลในการใช้ **Validation Set**

- ตรวจสอบโมเดลบนข้อมูลที่ *ไม่เคยเห็นมาก่อน*
- ใช้ดูว่าโมเดลเริ่ม **overfit** เมื่อไร
- ช่วยเลือกจำนวน **epochs** ที่เหมาะสม

```
x_val = x_train[:1000]  
partial_x_train = x_train[1000:]
```

```
y_val = one_hot_train_labels[:1000]  
partial_y_train = one_hot_train_labels[1000:]
```

Training the Model

Train 20 epochs พร้อมตรวจ validation ทุก epoch

```
history = model.fit(  
    partial_x_train,  
    partial_y_train,  
    epochs=20,  
    batch_size=512,  
    validation_data=(x_val, y_val))
```


Plot Training vs Validation Loss

Loss Curve = ตัวบอก overfitting ที่ชัดที่สุด

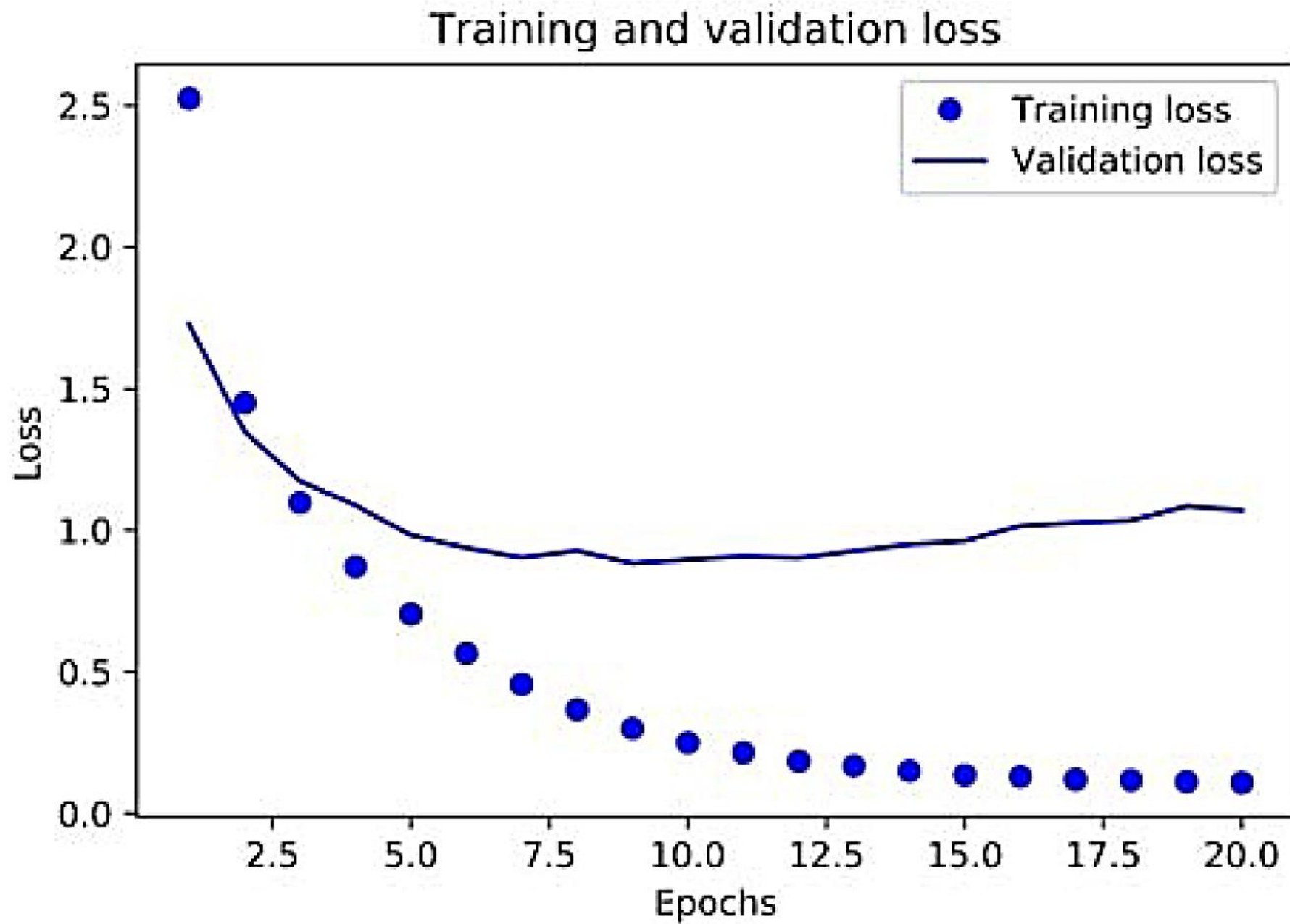
```
loss = history.history['loss']  
val_loss = history.history['val_loss']
```

```
import matplotlib.pyplot as plt
loss = history.history['loss']
val_loss = history.history['val_loss']

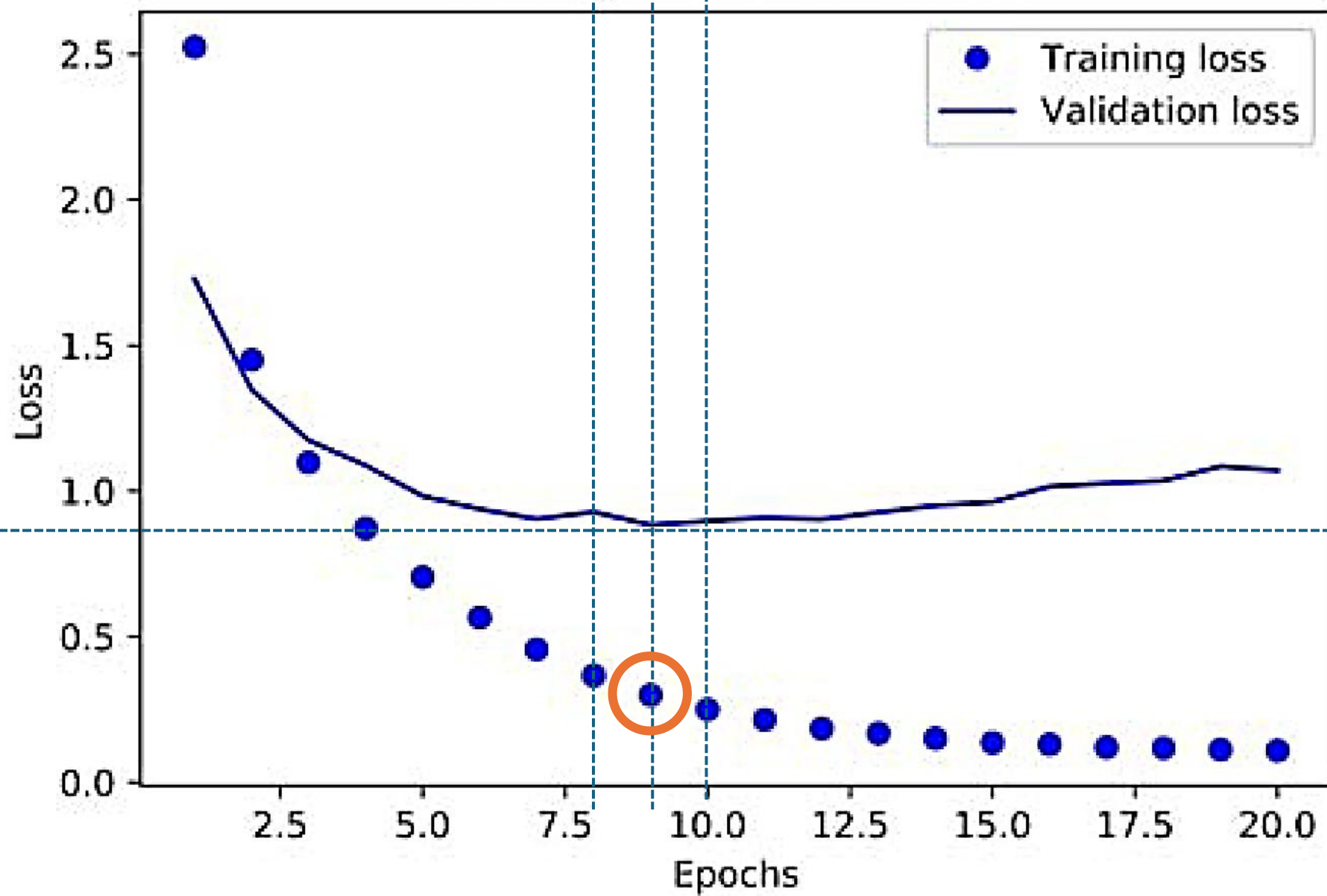
epochs = range(1, len(loss) + 1)

plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation
loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```



Training and validation loss



Plot Training vs Validation Accuracy

Accuracy Curve

- Training accuracy → สูงขึ้นเรื่อย ๆ
- Validation accuracy → ดีขึ้นช่วงแรก แล้วคงที่/ลดลง

```
acc = history.history['acc']
```

```
val_acc = history.history['val_acc']
```

Code ต่อจากข้างบน

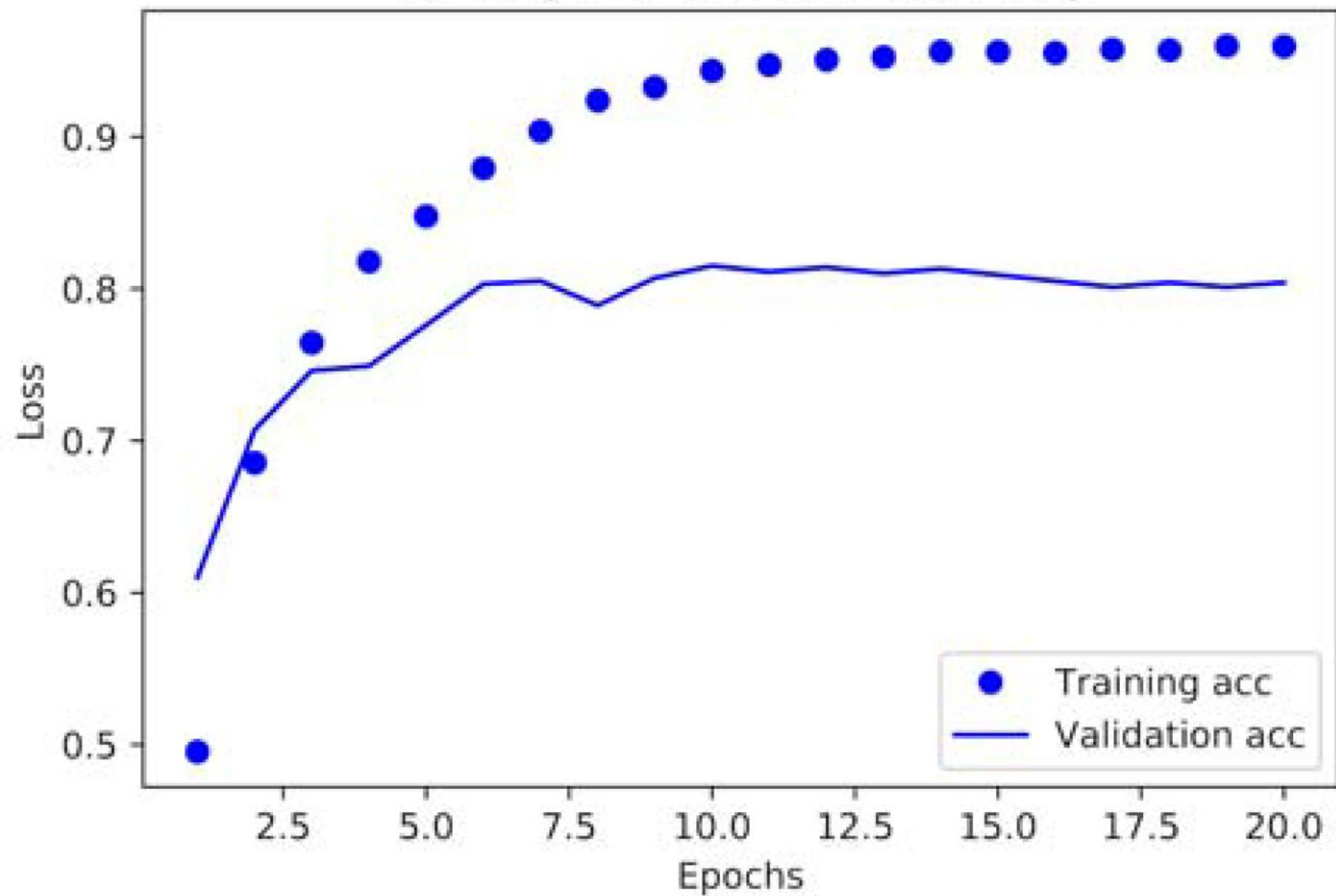
```
plt.clf()

acc = history.history['acc']
val_acc = history.history['val_acc']

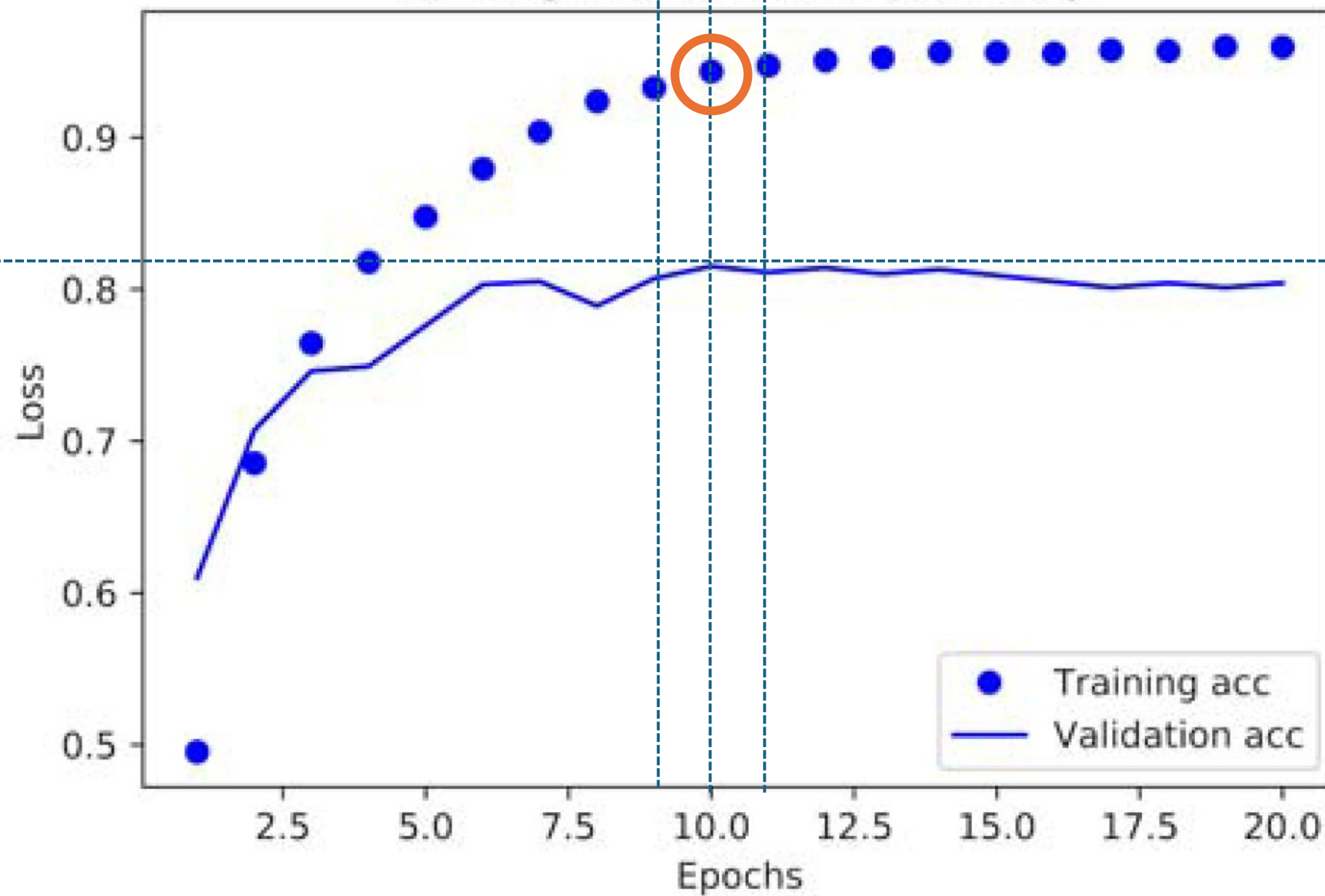
plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```

Training and validation accuracy



Training and validation accuracy



Retraining with Optimal Epochs

เหตุผลต้อง **retrain**

- จาก validation curves $\rightarrow$ optimal epochs ≈ 9
- Train ใหม่ตั้งแต่ต้นเพื่อให้ผลดีที่สุดบน test set

```
model.fit(partial_x_train, partial_y_train,  
          epochs=9, batch_size=512,  
          validation_data=(x_val, y_val))  
results = model.evaluate(x_test, one_hot_test_labels)
```

results

results

[0.9565213431445807, 0.79697239536954589]

- Accuracy ~80%

Comparing with Random Baselin

```
import copy
test_labels_copy = copy.copy(test_labels)
np.random.shuffle(test_labels_copy)
hits_array = np.array(test_labels) == np.array(test_labels_copy)
float(np.sum(hits_array)) / len(test_labels)
```

Random baseline = ~19% accuracy

การเดาสุ่ม(แท้ๆ)จะได้ผลประมาณ $1/46 \approx 2\%$

Validation

- แยก **validation set** เพื่อดู **overfitting**
- ยิ่ง **train** นาน **training loss** ยิ่งลด แต่ **validation loss** อาจเพิ่ม
- การเกิด **overfit** → ต้องลด **epochs**
- **Retrain** ด้วยจำนวน **epochs** ที่เหมาะสม
- เปรียบเทียบกับ **random baseline** เพื่อประเมินคุณภาพโมเดล